RL Based Ride-Sharing Approach for Joint Matching, Pricing, and Dispatching

Amara Pranav
Department of Artificial Intelligence
Amrita School of Artificial Intelligence, Bengaluru
Amrita Vishwa Vidyapeetham,India
bl.en.u4aid23003@bl.students.amrita.edu

B G Rajath Siddarth
Department of Artificial Intelligence
Amrita School of Artificial Intelligence, Bengaluru
Amrita Vishwa Vidyapeetham,India
bl.en.u4aid23006@bl.students.amrita.edu

V. Sidharrth
Department of Artificial Intelligence
Amrita School of Artificial Intelligence, Bengaluru
Amrita Vishwa Vidyapeetham,India
bl.en.u4aid23054@bl.students.amrita.edu

Paruchuri Sai
Department of Artificial Intelligence
Amrita School of Artificial Intelligence, Bengaluru
Amrita Vishwa Vidyapeetham,India
bl.en.u4aid23063@bl.students.amrita.edu

***Abstract*—Ride-sharing systems in big cities are confronted with significant challenges as they have to deal with dynamic ride requests, intelligent vehicle dispatching, passenger matching, route optimization, and adaptive pricing. Traditional heuristic methods typically are not scalable, don't optimize the utilization of fleets, and fail to balance supply and demand. To overcome these limitations, this work introduces a Distributed Reinforcement Learning (DRL) based ride-sharing system that combines the Deep Q-Network (DQN) dispatching, Demand-Aware Route Matching (DARM), and Distributed Pricing for Ride Sharing (DPRS). The proposed framework utilizes a customized grid-based ride sharing environment built upon demand patterns in NYC taxis. The city is divided into a 15 × 15 grid with 225 dispatch areas, and each vehicle commuting to these areas is considered an independent agent in the distributed multi-agent system. The dispatching problem is modeled as a Markov Decision Process (MDP) where DARM and DPRS are used to optimize insertion routes and to negotiate with the driver and/or customers while providing adaptive pricing. The proposed framework is experimentally validated and is shown to attain an acceptance rate of 91.2%, vehicle occupancy rate of 15.7%, average waiting time of 1.41 minutes and near zero idle vehicles fraction, while enhancing the overall utilization of the vehicle fleet and the drivers' profitability. The results indicate that the joint optimization of dispatching, matching, routing and pricing for ride sharing using Reinforcement Learning can effectively enhance the efficiency of ride sharing in a dynamic urban transportation system.**

***Index Terms*—Reinforcement Learning, Ride-Sharing, Deep Q-Network, Vehicle Dispatching, Route Optimization, Dynamic Pricing**

I. Introduction

With the growing need for mobility services to be affordable, flexible and efficient, ride-sharing has become a crucial part of modern urban transportation. Ride requests in Uber or Lyft are processed in large numbers in various areas of the city daily. With the increasing urbanization, real-time control of vehicle dispatching, passenger matching, route planning and ride pricing [7] have become more complex. In traditional ride-sharing systems, vehicles are inefficiently matched with riders in dynamic demand environments [14] with long wait times, suboptimal vehicle allocation, and low utilisation rates. Hence, it is essential to develop intelligent decision-making mechanisms to enhance the efficiency and quality of service in ride-sharing systems.

Ride requests are dynamically generated at various locations and times in large-scale urban settings and vehicles have limited capacity and availability. The major challenges of the current ride-sharing systems are to optimize the dispatching [8], passenger matching, routing, and pricing decisions simultaneously [9]. The restrictions often result in an imbalance in supply and demand; there are sometimes too many customers in one city region and too many vehicles in another. Besides, customers will usually want lower waiting times and good rates, while the drivers will want to get the best deal and steer clear of low demand destinations. A key challenge in an ITS is balancing these competing goals.

Traditional ride sharing methods are based on heuristic or static allocation strategies which are inadequate in highly dynamic environments. The typical methods lack the ability to forecast future demand for riding and cannot react in real time to changes in demand and traffic. One potential solution to this issue is the use of a technique called Reinforcement Learning (RL), which allows vehicles to learn an optimal dispatching strategy continuously by interacting with the environment. Specifically, DQN [10] can enable them to optimize dispatching decisions for long time horizons, while simultaneously leveraging reward-based optimization and future demand predictions and vehicle availability constraints. This allows for intelligent fleet balancing, decrease in idle cruising, and better customer service.

The overall goal for the proposed system is to create a distributed intelligent ride-sharing system that will be able to enhance the passenger–vehicle matching, route planning, and adaptive pricing, and maximize fleet utilization and driver profitability [11]. The main goals of the framework are to

decrease the waiting time for customers, increase the rate of ride requests being accepted, and enable scalable, real-time ride sharing in large urban areas. Multiple modules designed to optimize the vehicle dispatch, matching, routing and pricing jointly, and a scalable ride-sharing environment for urban dispatch simulation in an intelligent way.

The major contributions of this work are summarized as follows:

- A Distributed Deep Reinforcement Learning framework for intelligent vehicle dispatching in ride-sharing systems.
- A Demand-Aware Route Matching (DARM) module for efficient passenger matching and insertion-based route optimization.
- A Distributed Pricing for Ride Sharing (DPRS) module for adaptive pricing and customer-driver negotiation.
- A custom ride-sharing environment for scalable urban dispatch simulation.
- A multi-agent learning framework where each vehicle independently learns optimal dispatch policies.
- Joint optimization of dispatching, matching, routing, and pricing within a unified ride-sharing framework.

II. Related Work

Several research works have been proposed to improve ride-sharing systems through passenger matching, route optimization, dispatching, and pricing strategies. But current approaches still have serious limitations in efficiently dealing with large-scale dynamic urban environments.

The work presented in [1] is about approximation algorithms for assignment in ride-sharing. The proposed technique is aimed for two passengers per car, and primarily focuses on optimizing trip assignment. The ride sharing assignment problem is an NP-Hard problem and hence, the computational complexity becomes huge in large-scale real-time environment. Moreover, the concept does not include intelligent dispatch, dynamic pricing, or driver/customer preference management.

Heuristic allocation approaches for ride-sharing systems are introduced in the work [2]. The approached utilised is only able to accommodate the ride sharing with up to 3 people, but is mainly based on heuristic-based assignment techniques. With constantly evolving demand conditions the computational costs rise significantly, resulting in reduced scalability and flexibility in dynamic urban environments.

In [3], the authors address route optimization and shared mobility planning. The study emphasizes that existing approaches struggle with dynamic route optimization under changing demand conditions. Real-time optimization becomes quite challenging with increase in number of vehicles and ride requests.

The study [4] focuses mainly on pricing- aware allocation mechanisms for ride-sharing systems. The method takes into account price based optimization, but it does not optimize the dispatching, passenger matching and route planning simultaneously. In addition, it is challenging to achieve a balance between satisfaction and profitability for customers and drivers in large-scale real-time scenarios.

The paper [5] introduces insertion based routing techniques for dynamic ride-sharing services. But the method creates a group of passengers who are close together (with little regard for travel direction), leading to the possibility of opposite-direction passengers being grouped together. This leads to longer waiting times and distances for customers. It also does not include intelligent pricing and demand-aware dispatching features.

The authors of [6] propose a distributed dispatching algorithm based on deep reinforcement learning for ride-sharing systems. The approach promises to enhance the dispatch efficiency with reinforcement learning, but it does not include any adaptive pricing method or customer-driver decision negotiation. Matching, pricing and dispatching are not optimized simultaneously in a single framework.

Based on the literature, it can be noticed that most ride-sharing methods are limited to only one particular part of ride-sharing, which are dispatching, pricing or route optimization, independently. Very few systems work together to optimize intelligent dispatching, demand-aware route planning, adaptive pricing and customer-driver interaction in one system. Furthermore, computational complexity, scalability, imbalances in supply and demand, and dynamic real-time optimization are important challenges that still exist in current ride-sharing systems.

In order to overcome these limitations, this work introduces a Distributed Deep Reinforcement Learning-based ride-sharing framework that integrates three components, namely, intelligent DQN dispatching, Demand-Aware Route Matching (DARM), and Distributed Pricing for Ride Sharing (DPRS), under an integrated system architecture.

III. Proposed Methodology

The proposed framework proposes a Distributed Deep Reinforcement Learning (DDRL) based ride-sharing system to handle complex dispatching, passenger matching, route optimization and adaptive pricing of intelligent vehicles in urban areas. The system combines the dispatching optimization based on Deep Q-Network(DQN) with the optimization for Distributed Pricing for Ride Sharing(DPRS) and Demand-Aware Route Matching(DARM) to achieve the optimal utilization of a fleet, high quality of service and high driver profitability at the same time. This overall architecture of the proposed framework is described in Fig. 1. The system starts with customer rides requests in the central control unit, where the demand forecasting and ETA are done. DQN dispatching module sends idle vehicles to high-demand areas and DARM matches passengers and optimizes routes. Lastly, DPRS performs adaptive pricing and takes care of customer-driven negotiations before ride confirmation.

A. DQN Dispatching Module

The proposed framework includes the DQN Dispatching Module as a fundamental component of the Reinforcement Learning approach. The main goal of it is to route idle vehicles smartly to productive and potential hotspots in the city. The

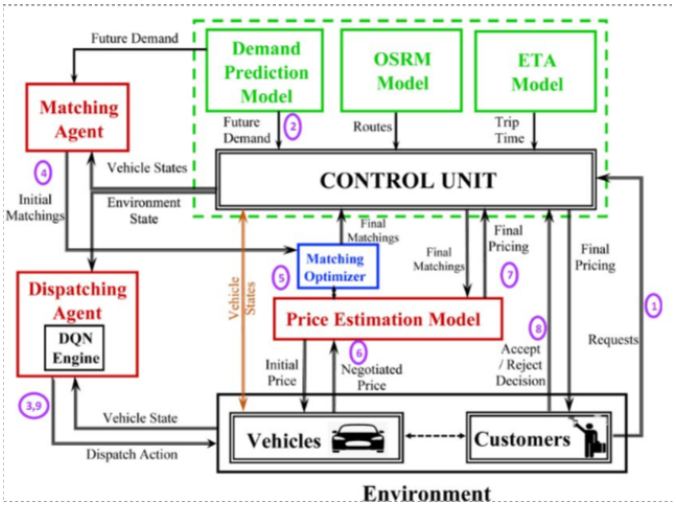

Fig. 1. Architecture diagram

proposed system is designed to allow vehicles to learn optimal dispatching strategies from continuous interaction with the ride-sharing environment instead of waiting for ride requests at random times. The dispatching problem is formulated as a Markov Decision Process (MDP) with the use of Deep Q-Networks (DQN) to learn long-term reward-maximizing policies.

1) Multi-Agent Distributed RL System: The proposed framework follows a distributed multi-agent Reinforcement Learning architecture.All the vehicles are individual RL agents carrying on a process of continuously observing the surrounding and choosing dispatching action according to expected future reward. The agents operate within the same urban ride-share system, but do not have direct communication between vehicles and learn their own dispatching policies.

The distributed learning framework has various benefits compared to centralized dispatching. Each car learns the dispatching policies on its own, so the system is quite scalable for large cities with thousands of cars and constantly changing ride requests.

Note that in the proposed framework only vehicles are considered RL agents. Consumers compare ride utility, based on price, waiting time, ride-sharing and vehicle type, and accept or reject ride requests.

2) Environment and Grid Representation: The proposed framework uses a custom ride-sharing simulation environment which has been developed based on the taxi dataset from New York City. Represents a dynamic urban transportation system with continuously changing ride requests, vehicle movements, passenger pickups and ride sharing interactions.

To make Reinforcement Learning computationally feasible, the city is broken down into several non-overlapping grid regions, with each grid cell comprising a single dispatch zone and corresponding to a 1-square-mile area. Rather than GPS coordinates, vehicles are sent between these larger cities.

For instance, instead of representing locations with very precise coordinates like (193, 267) and (193, 299), nearby regions are grouped into practical dispatch areas like downtown region, residential zone, airport zone, etc., which would result in very large continuous state spaces if trained with exact coordinates, and lead to an unfeasible computational cost and instability. If there are small differences in the coordinates, they can represent locations that are practically very close, but the RL model would still consider them as completely different states. Hence, the zone-based dispatching offers a feasible compromise between the computational efficiency, dispatch scalability and practicality. This environment is shown in Fig. 2.

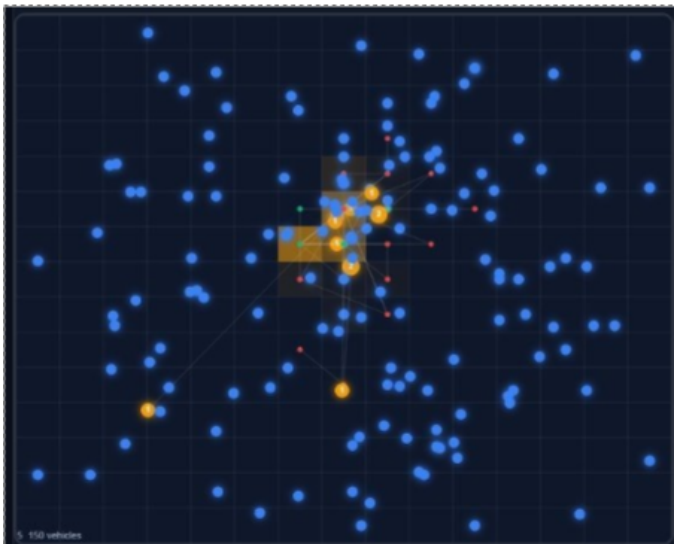

Fig. 2. The environment designed

3) Vehicle Representation: For each vehicle, the following information within the environment is maintained:

- Current zone/location
- Vehicle seating capacity
- Current occupied seats
- Passenger destinations
- Pickup schedules
- Vehicle availability status

These vehicle states are continually updated after each dispatching operation as well as ride sharing operation.

4) Dynamic Ride Requests: There are a number of customer ride requests that continuously enter into the environment and include Pickup location, Destination location, Passenger count, and Request time. These requests dynamically alter the distribution of demands in the city areas, requiring RL agents to constantly adjust the dispatching decisions.

5) Environment Constraints: There are also some operational constraints of the environment:

a) Capacity Constraint: Vehicles are assigned only if sufficient seating capacity is available.

b) Dispatch Radius Constraint: Vehicles are dispatched only in nearby areas to prevent unrealistic long-distance repositioning.

c) Ride Request Rejection Constraint: If there is no feasible vehicle in a certain radius of about 5 km, the request is rejected.

6) Custom Environment Limitations: While the environment is more scalable, there are some compromises made:

- Square dispatch zones are used to approximate exact road geometries. The actual traffic congestion is not completely modeled in real time.
- ETA estimation uses models and not real GPS traffic systems.
- Perception of Vehicle-to-vehicle communication is not taken into account.
- No explicit modelling of environmental factors like accidents, weather and road closures.
- Dispatched is zone based, not exact road level navigation.

The simplifications reduce the complexity of the computations and facilitate the scalable Reinforcement Learning training for large urban environments.

B. RL Formulation

The ride-sharing dispatching problem is formulated as a Markov Decision Process (MDP), where each vehicle agent learns an optimal dispatching policy through Reinforcement Learning.

1) Agent: There is an RL agent/player for each vehicle in the fleet. The agent is able to pick out the dispatching actions that maximize the long-term future reward by continuously observing the state of the environment.

2) Environment: Environment is the custom environment created in this work. All of the vehicle agents learn the dispatching policies independently, and interact in the same environment.

3) State Space: The state representation used in the framework is defined as:

$$s_t = (X_t, V_{t:t+T}, D_{t:t+T}) \tag{1}$$

where:

- X_t represents the current vehicle state information including location, seating capacity, passenger destinations, and pickup schedules.
- $V_{t:t+T}$ represents predicted future vehicle supply across different city zones.
- $D_{t:t+T}$ represents predicted future ride demand across city regions.

The state representation is used to learn dispatching policies based on the current environment conditions and expected future distribution of demand and supply.

4) Action Space: The action space is all possible possible dispatching movements that can be performed by an action space agent.

Vehicles navigate in the grid environment, rather than by GPS navigation, between the surrounding dispatch zones. For each action, the vehicle is moved from one zone to another in the immediate vicinity.

The framework allows for a horizontal and a vertical movement of a dispatch that does not exceed seven grid cells.

This gives an action space for dispatch as 15 x 15 with:

- The location of the current vehicle is denoted by the center cell.
- The surrounding cells are neighboring dispatch regions.

For example:

- moving left corresponds to dispatching toward western neighboring zones,
- moving right corresponds to eastern zones,
- moving upward corresponds to northern zones,
- moving downward corresponds to southern zones.

The action space can then be used to enable vehicles to move towards lucrative future demand areas intelligently, without an unrealistic long distance dispatch jump.

5) Reward Function: The reward function used by the framework is defined as:

$$r_{t,n} = \beta_1 C_{t,n} + \beta_2 T^D_{t,n} + \beta_3 T^E_{t,n} + \beta_4 P_{t,n} + \beta_5 \max(e_{t,n} - e_{t-1,n}, 0) \tag{2}$$

where:

- $C_{t,n}$ represents the number of customers served.
- $T^D_{t,n}$ represents dispatch time.
- $T^E_{t,n}$ represents additional travel delay caused by ride-sharing.
- $P_{t,n}$ represents driver profit.
- The final term represents fleet utilization improvement.

The reward function jointly optimizes customer service quality, driver profitability, dispatch efficiency, and fleet utilization.

C. DQN Neural Network Architecture

The proposed framework is based on the Deep Q-Network (DQN) which is an algorithm that approximates an optimal dispatching policy. The state vector for the environment is fed into the DQN, which then predicts the Q- values of the actions that can be dispatched.

The input layer takes the following:

- Current vehicle state
- Future supply prediction
- Future demand prediction

The deep layers of the neural network are learning complex relationships between vehicle states, demand distribution, and long-term dispatch. The output layer is the layer that computes Q values for all possible dispatch actions.

For a given state-action pair (s, a), the DQN estimates:

$$Q(s, a) \tag{3}$$

which represents the expected future reward obtained by taking action a in state s.

The vehicle selects the dispatch action corresponding to the maximum Q-value:

$$\arg\max Q(s, a) \tag{4}$$

During training phase, experience replay memory is used to promote stability of learning. The previous experiences of (s_t, a_t, r_t, s_{t+1}) are stored in replay memory and retrieved at random during training. The Bellman update equation is used to update the DQN parameters:

$$Q(s_t, a_t) \leftarrow r_t + \gamma \max Q(s_{t+1}, a') \tag{5}$$

where γ represents the discount factor used to balance immediate and future rewards.

D. DQN Learning Workflow

The learning workflow starts with the vehicle agent observing the current state of the environment. The state vector observed is fed into the DQN along with the Q-values that the DQN predicts for each of the possible dispatch actions. The vehicle then chooses the action to dispatch based on the highest Q-value predicted and travels towards the chosen zone. Once the dispatch action is performed, the environment is able to calculate the reward based on: customers served, dispatch delay, driver profit, and fleet utilization. The new state, reward are stored in replay memory, and the DQN parameters are updated by the Bellman equation. The vehicle learns to dispatch in an optimal way, maximizing long-term future reward, by repeatedly interacting with its environment.

IV. DARM Module

The proposed ride-sharing system has a module called the Demand-Aware Route Matching (DARM) module that is tasked with passenger–vehicle matching and route optimization.

A. Demand-Aware Route Matching

The DARM framework conducts passenger matching with route-awareness, taking both spatial and temporal compatibility into consideration for ride requests. The framework will not only assign passengers according to the shortest distance but also try to group the passengers with similar travel directions and similar routes. The framework is based on future demand information produced by the DQN Dispatching Module to prevent the assignment of vehicles to routes that could lead to a decrease in the future availability of services in a high demand area. The matching process tries to maximize ride-sharing, minimize any unnecessary detours, lessen customer wait time, increase the vehicle occupancy and optimize the overall route efficiency. Avoiding inefficient opposite direction traveling passenger ride-share combinations, the proposed DARM framework uses route-aware matching and insertion-based optimization.

B. Greedy Matching Algorithm

The first step of DARM involves a greedy passenger–vehicle assignment. The goal of the greedy matching algorithm is to efficiently match a new ride request to a suitable vehicle, given some operational constraints.

Once a new request for ride is received, the framework scans the service area for the candidate vehicles that are close to the request and within a certain dispatch radius (around 5 km) which is pre-defined and is not considered as a feasible vehicle candidate for ride assignment.

The possibility of the vehicle being feasible can only be determined if adequate seating is available [13]. The capacity constraint is:

$$V_C + |r_i| \leq C_{max} \tag{6}$$

where:

- V_C represents the current occupied seating capacity of the vehicle,
- $|r_i|$ represents the number of passengers in request r_i,
- and C_{max} represents the maximum vehicle seating capacity.

Once the vehicles that are feasible candidates have been identified, the framework estimates the expected arrival time of the vehicles in the pickup zone, based on the ETA prediction model. The first car to use is the one with the shortest estimated pick up time.

But greedy assignment is not enough to make sure the resulting route is globally optimal. Even with the new system, passengers could be assigned to sub-optimal ride sharing routes to extend the travel delay and detour distance. Hence, the framework implements the insertion-based route optimization after initial assignment.

C. Insertion-Based Route Planning

Initial passenger assignment is followed by a ride-sharing efficient route optimization based on insertion, performed by the DARM framework. The insertion algorithm is an attempt to add new passenger pick up and drop off points to the already established vehicle routes, and to minimise any extra travel cost and delay.

Suppose the current route of vehicle V_j is represented as:

$$S_{V_j} = [r_1, r_2, ..., r_k] \tag{7}$$

Once a new ride request comes in, the framework considers various possible insertion points for the new passenger in the original route.

The route optimization process follows the steps below:

1) Get the existing route of the assigned vehicle.
2) Place the new passenger pick-up and drop-off sites in various reasonable routing locations.
3) Calculate the extra route cost of each insertion configuration.
4) Choose the route configuration that has the lowest incremental travel cost.

The travel distance, expected travel delay, feasibility of route compatibility and seats are used to calculate the additional route cost. This framework is based on the Open Source Routing Machine (OSRM) software to generate shortest paths and estimate road distances realistically. By using OSRM to calculate practical urban routes, the system is additionally able to calculate routes that have a more realistic Euclidean

distance approximation.The computation of practical urban routes allows the system to calculate routes with a more realistic Euclidean distance approximation besides Euclidean distance approximation.

The DARM module, hence, allows scalable and efficient passenger matching and route optimization for a dynamic ride sharing system in a large urban transportation environment.

V. DPRS Module

Within the proposed framework, adaptive ride pricing and driver–customer negotiation is addressed by the Distributed Pricing for Ride Sharing (DPRS) module. Once the passenger matching and route optimization are finished by the DARM module, the DPRS module calculates the passenger's appropriate ride price based on the cost of the route, fuel consumption, waiting times, ride-sharing level, and the future demand conditions of the route.

A. Distributed Pricing Framework

The distributed pricing [12] mechanism takes into account the distance traveled in the current trip; the amount of fuel expected to be used; waiting time of the customer; the level of ridesharing; and the ride demand of the customer anticipated for the future. DPRS's main goal is to ensure customers' affordability, drivers' profitability and overall efficiency of ride sharing. The pricing system also aims to mitigate ride rejection by making pricing of rides and choice of destination more fair.

B. Initial Ride Pricing

The framework calculates an initial ride cost first from the operational ride cost and ride sharing conditions. The basic price of a ride is set to be:

$$P_{init} = B_j + \left(\omega_1 \frac{cost}{V_C}\right) + (\omega_2 \times fuel\_cost) - (\omega_3 \times wait\_time) \tag{8}$$

where:

- B_j represents the base earning of driver j,
- $cost$ represents the total route cost associated with ride request r_i,
- V_C represents the number of passengers sharing the ride,
- ω_1, ω_2, and ω_3 are weighting parameters,
- $fuel\_cost$ represents estimated fuel consumption cost,
- and $wait\_time$ represents customer pickup waiting time.

The pricing formulation tries to allocate a fair share of the route cost to passengers, and also include operational driving costs and customer waiting penalties.

There are a few important factors that affect the ride price:

- Ride price is typically higher the farther away the journey.
- The higher the fuel consumption, the higher the operational cost.
- Longer wait time impacts utility of the ride and can lower acceptable pricing.
- Multiple passengers riding in a vehicle decreases the cost per passenger.

C. Driver Price Adjustment

Once the initial ride price is produced, a driver determines on his own if the suggested ride is profitable. The DQN Dispatching Module gives future reward estimate for various city areas. This future demand information is used by drivers to decide if it is economically worthwhile for them to travel towards a specific destination region. If the destination is in a high demand zone, the driver might be willing to take the offer, reasoning that if that's the case, there will be more rides in the future in the immediate vicinity. Drivers may raise the ride price if the destination is in a low demand or isolated area, so as to secure more rides in the future than they would otherwise receive and reduce the idle distance traveled.

D. Customer Utility Function

Once the price of the ride is decided, the customer is challenged to determine if the ride is recommended as meeting his/her utility preferences. The utility to the customer is related to ride-sharing, waiting time and vehicle quality.

The customer utility function is given by:

$$U_i = \left(\omega_4 \times \frac{1}{V_C}\right) + \left(\omega_5 \times \frac{1}{T_i}\right) + (\omega_6 \times V_T) \tag{9}$$

where:

- V_C represents ride-sharing level,
- T_i represents customer waiting time,
- V_T represents vehicle type,
- and ω_4, ω_5, and ω_6 are weighting parameters.

The utility function reflects some customer preferences:

- The shorter a person has to wait for a service, the more their utility is increased.
- The improved quality of the vehicles leads to higher utility.
- Reduce crowding effects in the ride-sharing system to make customers more comfortable.

E. Accept/Reject Decision Logic

The relationship between the utility and ride price is what is used to determine whether a customer accepts or rejects a ride request.

If the customer accepts the ride, then:

$$U_i > P(r_i) - \delta_i \tag{10}$$

where:

- $P(r_i)$ represents the final ride price,
- and δ_i represents the customer compromise threshold.

If the ride request is accepted by the customer when customer utility is greater that ride price is effective, then the ride is accepted, else it is rejected.

The DPRS module is thus an important element in the proposed framework and facilitates intelligent pricing and negotiation in a dynamic urban ride sharing environment.

VI. IMPLEMENTATION DETAILS

A. System Implementation

The proposed ride-sharing framework has been implemented in python and designed as a modular Reinforcement Learning-based transportation system that integrates DQN dis- patching, Demand-Aware Route Matching (DARM) and Distributed Pricing for Ride Sharing (DPRS). The implementation is an extension to the theoretical framework suggested in the base paper, creating a fully functioning simulation environment which provides real-time experimentation and monitoring support.

The complete framework was implemented using three major components:

- `ridesharing_darm_dprs_dqn.py`
- `api_server.py`
- `test_api.py`

The custom ride-sharing environment, Double DQN dispatching module, DARM route optimization module, DPRS adaptive pricing system, congestion simulation and evaluation pipeline were incorporated in a single overall structure in the main simulation engine, called `ridesharing_darm_dprs_dqn.py`,

A fully custom ride-sharing environment was developed instead of using existing simulation libraries such as OpenAI Gym or SUMO. A 15 × 15 grid structure with 225 dispatch zones was used to represent the city environment. This is a good approximation of the zone area, which is equivalent to a small urban area of approximately 0.8 km, in order to lower the state space dimension and improve the scalability of Reinforcement Learning. Environment is constantly updated: dynamic customer ride requests, vehicle location, passenger occupancy, pick up schedules, route assignments.

The framework also represents several operational limitations such as: vehicle seating capacity, dispatch radius limitations, waiting-time limitations and ride reject conditions.

For the sake of achieving realism in simulation, the congestion-aware routing was introduced, considering the stochastic traffic condition. Additional rider-behaviour simulation has been achieved by: random canceling of rides, changing ride destinations and changing ride-demand distributions. The proposed framework is based on a distributed multi-agent Reinforcement Learning (RL) architecture with each vehicle being considered as an independent RL agent. The policy network of all vehicles, however, is shared, and that is a Double DQN policy network, which is scalable for learning across the fleet.

B. API Framework and Dashboard System

The system architecture was designed to be API-based, with the`api_server.py`module used to make the components of the system independent of each other, specifically Reinforcement Learning training, simulation execution, visualization, and experimentation control. The modular design enhances the scalability, maintainability, and asynchrony of the RL framework. There are several REST endpoints provided by the API server for: RL training control, model management, simulation monitoring and evaluation.

The API server exposes multiple REST endpoints for: RL training control, model management, simulation monitoring,and evaluation.

Training APIs support: start training, pause/resume training, stop training and retrieve training status.

Model management APIs allow to save trained models, load trained models, get information about saved models, and delete old experiments.

Vehicle states, ride requests, occupancy, route schedules, congestion statistics, and more are continuously provided through simulation APIs.

For real-time monitoring, it leverages Server-Sent Events (SSE) to stream the metrics continuously from the RL engine to the dashboard interface. Real-time visualization of moving vehicles, demand heat maps, route schedules, ride requests and training metrics on the dashboard.

The framework continuously monitors many important Reinforcement Learning and transportation-system metrics such as the acceptance rate, driver profit, occupancy rate, waiting time, replay buffer utilization, DQN loss, and Q-value convergence.

C. Training Details

A Double Deep Q-Network (Double DQN) was used to train the dispatching problem. The DQN is given the state vector of the environment as input, and outputs Q-values for possible dispatch actions.

The DQN training process follows the workflow below:

1) Generate dynamic ride requests.
2) Perform DARM passenger matching.
3) Apply DPRS adaptive pricing.
4) Execute customer accept/reject decision.
5) Select DQN dispatch action.
6) Compute environment reward.
7) Store transition in replay memory.
8) Update Double DQN parameters.

The framework implements epsilon-greedy exploration in training. An adaptive exploration mechanism was also added which automatically increases the value of epsilon if the acceptance rate is not improving over a longer period of time. That avoids poor local policies and helps to keep the discussion going.

The final training configuration used in the implementation is shown in Table I.

The reward function jointly optimizes:

- customer service quality,
- driver profitability,
- dispatch efficiency,
- and fleet utilization.

The reward configuration used in the implementation is shown in Table II.

The proposed implementation, therefore, offers a fully deployable Reinforcement Learning experimentation framework

TABLE I
DQN TRAINING HYPERPARAMETERS

Hyperparameter	Value
Learning Rate	$5e^{-4}$
Discount Factor (γ)	0.95
Replay Buffer Size	5000
Batch Size	64
Epsilon Start	1.0
Epsilon End	0.05
Epsilon Decay	0.997
Target Network Update	150

TABLE II
REWARD WEIGHT CONFIGURATION

Reward Component	Weight
Customers Served	+10
Dispatch Time	-1
Detour Time	-5
Driver Profit	+12
Idle Vehicle Penalty	-8

for intelligent optimization of ride-sharing in large urban transportation environments.

VII. RESULTS AND ANALYSIS

A. Evaluation Metrics

The proposed framework was tested with various Reinforcement Learning (RL) and transportation system performance metrics to investigate dispatch efficiency, ride sharing quality, economic performance and utilization of the transportation fleet. The experiments use the following summary of evaluation metrics of Table III.

TABLE III
EVALUATION METRICS

Metric	Description
Acceptance Rate	Ratio of accepted ride requests to total requests generated
Driver Profit	Total cumulative profit earned by drivers
Waiting Time	Average passenger pickup delay
Occupancy Rate	Percentage of vehicle capacity utilized
Idle Fraction	Percentage of idle vehicles in the fleet
Reward	Reinforcement Learning reward convergence quality

Continuous monitoring of the metrics was done during training using the real-time dashboard system implemented via the API framework. The evaluation was conducted with dynamic ride sharing conditions such as stochastic congestion, fluctuating customer demand, changing rider destinations and ride cancellations.

B. Experimental Results

The entire DQN + DARM + DPRS framework was trained for 1500 training steps and evaluated for 300 evaluation steps. The result of the implemented system is given in Table IV below.

The experimental results show the proposed framework is able to learn intelligent dispatching behavior with Double DQN. Gradual reconfiguration of vehicles towards profitable demand areas, eliminating unnecessary idling and balancing supply and demand across the city grid. The evaluation metrics are shown in Fig. 3. Route optimization using the DARM insertion method is able to reduce the empty spaces on the bus and the number of stops per route by a clever combination of passengers' travel directions. The proposed route optimization strategy can be compared with the traditional greedy matching strategies, both of which enhance the quality of ride sharing and improve the efficiency of passenger grouping.

TABLE IV
FINAL PERFORMANCE OF PROPOSED FRAMEWORK

Metric	Obtained Result
Acceptance Rate	91.2%
Driver Profit	$1194.89
Average Wait Time	1.41 min
Occupancy Rate	15.7%
Idle Fraction	0.0%

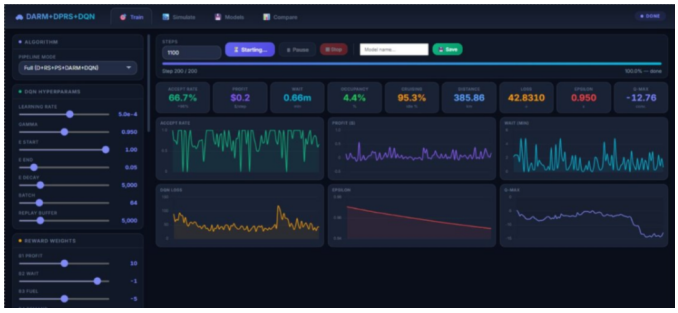

Fig. 3. The evaluation

The DPRS adaptive pricing module is able to dynamically adjust the price of a ride based on demand, route cost, future destination profitability, and congestion. While dynamic pricing crudely affects ride acceptance in some situations, it significantly increases overall driver productivity and profitability and economic efficiency.

The experimental analysis also shows that proactive DQN dispatching greatly reduces the idle fraction of the vehicle fleet. They constantly shift their fleets to anticipated hotspot locations which leads to improved fleet utilization and reduced wait times.

C. Rider Change Sensitivity Analysis

To test the stability of the framework under customer trip behaviour, further experiments were carried out by adding trip destination changes and ride cancellations during customer trips. The probability of a rider change was systematically raised, and the acceptance rate, profit and waiting time were measured. The results are summarized in Table V.

The system is stable even with more rider disturbance. The overall degradation gets more and more shallow as the destination change probability increases, but it is still a relatively small process as the rate of acceptance goes down. This shows that the proposed ride sharing framework is robust

TABLE V
RIDER CHANGE SENSITIVITY ANALYSIS

Dest. Change Probability	Acc. Rate	Profit	Wait Time (min)
0%	94.9%	21.07	1.16
3%	91.2%	20.94	1.22
6%	87.5%	20.61	1.31
10%	81.7%	19.94	1.46

enough against unpredictable customer behaviour and dynamic changes in routes.

D. Ablation Study

To test each of the proposed framework components separately, an ablation study was performed. DQN dispatching, DARM route optimization, DPRS pricing and ride-sharing functionality were all removed and experiments were conducted. The ablation results are summarized in Table VI.

TABLE VI
ABLATION STUDY RESULTS

Configuration	Accept Rate	Profit	Wait Time	Occupancy
Full System	94.9%	21.07	1.16	4.78%
Without Dispatch	70.7%	20.73	1.29	4.99%
Without Ride-Sharing	94.1%	23.58	1.28	5.23%
Without DARM	88.2%	24.58	1.11	5.27%

The ablation study clearly proves that improvement in the acceptance rate and balancing supply and demand is the most critical part of the dispatching system and is achieved by using DQN. By eliminating proactive dispatching, ride throughput is substantially lowered and fleet imbalance is increased. The DARM route optimization module optimizes the routes to reduce inefficient detours and increases the efficiency of grouping passengers. DRPS pricing mechanism brings adaptive economic optimization. Slightly accepting the removal of pricing will increase the acceptance rate, but the system will no longer have the intelligent profitability optimization and dynamic pricing control.

Overall, the competitive combination of DQN + DARM + DPRS provides a better trade-off between ride acceptance, economic efficiency, fleet utilization and ride-sharing quality.

VIII. CONCLUSION

This work put forward a Distributed Reinforcement Learning-Dispatching (DRL), a Demand-Aware Route Matching (DARM), and Distributed Pricing for Ride Sharing (DPRS) algorithm, and proposed a combination of the three in a single system for ride sharing. The framework optimizes collective vehicle dispatch, passenger matching, route planning, and flexible pricing in dynamic urban settings.

A custom ride-sharing simulation environment is created based on grid-based city representation, Ride Request generation with dynamic load, Congestion-aware routing and Multi-agent dispatching. The DQN Dispatching Module optimised supply-demand balancing and decreased the idle movement of vehicles, the DARM module saved the vehicle occupation and optimised the route based on the insertion idea. Adaptive pricing and customers-driven negotiation were implemented to increase economic efficiency (DPRS module).

Experimental results have shown an improvement in the efficiency of ride sharing, fleet utilization, occupancy, and profitability of the drivers. The ablation study also validated the significance of dispatching and route optimization and adaptive pricing within the suggested framework.

Future research will involve implementing live GPS traffic information, Graph Neural Networks (GNNs), federated multi- agent Reinforcement Learning and autonomous vehicle coordination for the deployment of large-scale real-world ride-sharing applications.

REFERENCES

[1] X. Bei and S. Zhang, "Algorithms for trip-vehicle assignment in ride-sharing," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.

[2] J. Alonso-Mora, S. Samaranayake, A. Wallar, E. Frazzoli, and D. Rus, "On-demand high-capacity ride-sharing via dynamic trip-vehicle assignment," *Proceedings of the National Academy of Sciences*, vol. 114, no. 3, pp. 462–467, 2017.

[3] Y. Tong, Y. Zeng, Z. Zhou, L. Chen, J. Ye, and K. Xu, "A unified approach to route planning for shared mobility," *Proceedings of the VLDB Endowment*, vol. 11, no. 11, p. 1633, 2018.

[4] M. Asghari, D. Deng, C. Shahabi, U. Demiryurek, and Y. Li, "Price-aware real-time ride-sharing at scale: An auction-based approach," in *Proceedings of the 24th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, pp. 1–10, 2016.

[5] Y. Xu, Y. Tong, Y. Shi, Q. Tao, K. Xu, and W. Li, "An efficient insertion operator in dynamic ridesharing services," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 8, pp. 3583–3596, 2020.

[6] A. O. Al-Abbasi, A. Ghosh, and V. Aggarwal, "Deeppool: Distributed model-free algorithm for ride-sharing using deep reinforcement learning," *IEEE Transactions on Intelligent Transportation Systems*, vol. 20, no. 12, pp. 4714–4727, 2019.

[7] J. Huang, L. Huang, M. Liu, H. Li, Q. Tan, X. Ma, J. Cui, and D.-S. Huang, "Deep reinforcement learning-based trajectory pricing on ride-hailing platforms," *ACM Transactions on Intelligent Systems and Technology (TIST)*, vol. 13, no. 3, pp. 1–19, 2022.

[8] Y. Liu, F. Wu, C. Lyu, S. Li, J. Ye, and X. Qu, "Deep dispatching: A deep reinforcement learning approach for vehicle dispatching on online ride-hailing platform," *Transportation Research Part E: Logistics and Transportation Review*, vol. 161, p. 102694, 2022.

[9] Z. Zhang, L. Yang, J. Yao, C. Ma, and J. Wang, "Joint optimization of pricing, dispatching and repositioning in ride-hailing with multiple models interplayed reinforcement learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 12, pp. 8593–8606, 2024, doi: 10.1109/TKDE.2024.3464563.

[10] Y. Jiao, X. Tang, Z. T. Qin, S. Li, F. Zhang, H. Zhu, and J. Ye, "Real-world ride-hailing vehicle repositioning using deep reinforcement learning," *Transportation Research Part C: Emerging Technologies*, vol. 130, p. 103289, 2021.

[11] X. Tang, Z. Qin, F. Zhang, Z. Wang, Z. Xu, Y. Ma, H. Zhu, and J. Ye, "A deep value-network based approach for multi-driver order dispatching," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 1780–1790, 2019.

[12] Z. Zhang, "Dynamic pricing strategy optimization based on a reinforcement learning PPO algorithm: An empirical study on ride-hailing platforms," *Journal of Organizational and End User Computing (JOEUC)*, vol. 38, no. 1, pp. 1–43, 2026.

[13] J. Alonso-Mora, S. Samaranayake, A. Wallar, E. Frazzoli, and D. Rus, "On-demand high-capacity ride-sharing via dynamic trip-vehicle assignment," *Proceedings of the National Academy of Sciences*, vol. 114, no. 3, pp. 462–467, 2017.

[14] G. Guo and Y. Xu, "A deep reinforcement learning approach to ride-sharing vehicle dispatching in autonomous mobility-on-demand systems," *IEEE Intelligent Transportation Systems Magazine*, vol. 14, no. 1, pp. 128–140, 2022, doi: 10.1109/MITS.2019.2962159.